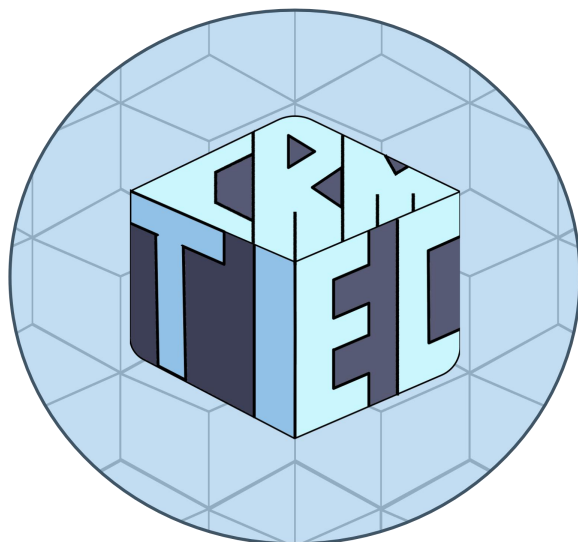


**M-Tec bajo la guía de la Universidad Tecnológica de Tecámac
PRESENTA:**



Manual Técnico para el manejo de CRM-Tec

Escrito por:

- CRISTIAN ALEJANDRO GONZALEZ CAMACHO**
- EDUARDO MONTOYA DUEÑAS**
- IRVIN DAVID REYES ROMANO**
- ALEXIS SANCHEZ AQUINO**
- IAN DANIEL VERA HERNÁNDEZ**

ÍNDICE

CAPÍTULO 1. REQUERIMIENTOS TÉCNICOS

Requerimientos de Hardware

1.2 Requerimientos de Software

1.3 Requerimientos de Red y Conectividad

CAPÍTULO 2. HERRAMIENTAS UTILIZADAS PARA EL DESARROLLO

2.1 Entorno de Desarrollo Integrado (IDE): Visual Studio Code

2.2 Framework de Interfaz Gráfica: JavaFX

2.3 Constructor de Interfaces Visuales: Scene Builder

2.4 Backend as a Service (BaaS) y Base de Datos: Supabase

3. CONFIGURACIÓN DEL APLICATIVO

3.1 Configuración de API Keys y Servicios Externos

3.1.1 Supabase

3.1.2 Google OAuth 2.0 y Firebase

3.1.3 API de Unsplash

3.2 Configurar la base de datos

3.2.1 Requisitos Previos

3.2.2 Parámetros de Conexión

3.2.3 Procedimiento de Configuración

3.2.4 Validación de la Conexión

CAPÍTULO 4. CASOS DE USO

CU-01: Inicio de Sesión Local

CU-02: Inicio de Sesión con Plataformas Externas SSO

CU-03: Creación de Usuario y Registro de Negocio

CU-04: Visualización de Inventario y Enriquecimiento Visual

CU-05: Registro de Nuevo Producto

CU-06: Gestión de Agenda y Calendario Interactivo

CU-07: Visualización, Búsqueda y Gestión de Estado de Tareas

CU-08: Registro y Asignación de Nueva Tarea

CU-09: Visualización, Búsqueda y Filtrado de Contactos

CU-10: Registro de Nuevo Contacto

CU-11: Interacción con el Asistente Inteligente

CAPÍTULO 5. MÓDULO DE ADMINISTRACIÓN

5.1 Gestión de Roles y Privilegios

5.2 Configuración del Perfil Empresarial (Negocio)

5.3 Segmentación de Espacios de Trabajo (Data Isolation)

CAPÍTULO 6. Modelo Entidad-Relación

CAPÍTULO 7. DICCIONARIO DE DATOS DEL MODELO ENTIDAD-RELACIÓN

CAPÍTULO 8. PROTOTIPOS DE PANTALLAS DEL APLICATIVO

8.1 Pantalla de Inicio de Sesión (loginM.fxml)

8.2 Pantalla de Registro de Usuario

8.3 Pantalla de Registro de Negocio

8.4 Pantalla Principal (Dashboard)

8.5 Pantalla de Inventario

8.6 Pantalla de Contactos (Directorio)

8.7 Pantalla de Tareas

8.8 Pantalla del Asistente IA (Módulo Gemini)

CAPÍTULO 1. REQUERIMIENTOS TÉCNICOS

Para garantizar la correcta ejecución, fluidez y seguridad del aplicativo CRM-Tec, el entorno de despliegue (equipo del usuario final) debe cumplir con los siguientes requerimientos mínimos y recomendados.

Debido a que el procesamiento de datos y el almacenamiento se delegan a una infraestructura en la nube (BaaS), las exigencias de hardware local son moderadas, priorizando la capacidad de red y la memoria volátil.

1.1 Requerimientos de Hardware

Componente	Requisito Mínimo	Requisito Recomendado	Justificación Técnica
Procesador (CPU)	Intel Core i3 (3ª Gen) / AMD Ryzen 3 o equivalente (Dual-Core a 2.0 GHz).	Intel Core i5 / AMD Ryzen 5 o superior (Quad-Core).	El sistema ejecuta hilos en segundo plano para consultas SQL y renderiza un motor web embebido (WebView) para la agenda interactiva.
Memoria RAM	4 GB	8 GB	La Máquina Virtual de Java (JVM) y el motor WebKit consumen memoria de forma dinámica al cargar la cuadrícula de inventario con imágenes externas.
Almacenamiento	500 MB de espacio disponible (HDD).	1 GB de espacio disponible en Disco de Estado Sólido (SSD).	La aplicación no almacena bases de datos locales; el espacio es exclusivamente para el instalador, la JRE y archivos temporales de caché visual.

Componente	Requisito Mínimo	Requisito Recomendado	Justificación Técnica
Resolución de Pantalla	1024 x 768 píxeles	1366 x 768 píxeles o superior.	Necesario para visualizar correctamente el GridPane completo y las tablas de datos sin recortes en la interfaz de usuario.

1.2 Requerimientos de Software

- Sistema Operativo: Windows 10 / Windows 11 (Arquitectura de 64 bits), macOS 10.14 Mojave o superior, o distribuciones Linux modernas con interfaz gráfica.
- Entorno de Ejecución: Java Runtime Environment (JRE) o Java Development Kit (JDK) versión 11 o superior. Nota: Esto es estrictamente necesario ya que el sistema utiliza la librería nativa `java.net.http.HttpClient` para comunicarse con las APIs de Gemini y Supabase, la cual no está disponible en versiones anteriores a Java 11.
- Navegador Web: Un navegador web predeterminado configurado en el sistema operativo (Chrome, Edge, Firefox, Safari) para permitir la redirección de los protocolos de autenticación SSO (Google/GitHub).

1.3 Requerimientos de Red y Conectividad

- Conexión a Internet: Conexión de banda ancha permanente (Min. 5 Mbps). El sistema quedará inoperante sin red, ya que depende de conexiones JDBC remotas, Firebase, Supabase y la API de Unsplash.
- Disponibilidad de Puertos: El Puerto Local 8080 del equipo debe estar desbloqueado y libre de uso. El aplicativo levanta temporalmente un servidor local (`HttpServer` / `LocalServerReceiver`) en este puerto para interceptar los tokens de seguridad durante el inicio de sesión OAuth.

CAPÍTULO 2. HERRAMIENTAS UTILIZADAS PARA EL DESARROLLO

Para la construcción del aplicativo CRM-Tec, se optó por un ecosistema de desarrollo moderno que permite una clara separación entre la lógica de negocio, el diseño de la interfaz de usuario y la persistencia de datos en la nube. A continuación, se detallan las herramientas tecnológicas fundamentales empleadas en el ciclo de vida del proyecto.

2.1 Entorno de Desarrollo Integrado (IDE): Visual Studio Code

Para la escritura del código fuente, estructuración del proyecto y depuración, se empleó Visual Studio Code (VS Code).

Justificación técnica: Se eligió por ser un editor de código ligero, multiplataforma y altamente personalizable. Mediante la instalación del Extension Pack for Java, VS Code proporcionó soporte completo para la compilación, autocompletado inteligente (IntelliSense) y gestión de las dependencias necesarias para ejecutar los módulos del CRM y los servidores locales de interceptación de tokens.

2.2 Framework de Interfaz Gráfica: JavaFX

El frontend de la aplicación de escritorio fue desarrollado íntegramente utilizando la plataforma JavaFX.

Justificación técnica: JavaFX proporciona una biblioteca robusta de controles de interfaz de usuario (UI) y gráficos acelerados por hardware. Su implementación permitió construir una experiencia de usuario fluida y reactiva. Además, su componente WebView y WebEngine fueron fundamentales para incrustar e interpretar código HTML y JavaScript dentro de la aplicación nativa, lo cual hizo posible la renderización de la agenda interactiva bidireccional.

2.3 Constructor de Interfaces Visuales: Scene Builder

Para agilizar el diseño de las pantallas del aplicativo, se utilizó Scene Builder, una herramienta de diseño visual tipo Drag-and-Drop (arrastrar y soltar) desarrollada originalmente por Oracle y mantenida por Gluon.

Justificación técnica: Scene Builder permite maquetar las vistas gráficas sin necesidad de escribir el código de marcado manualmente. Al diseñar las pantallas (como el panel de control o los formularios de registro), la herramienta genera automáticamente los archivos con extensión .fxml (ej. loginM.fxml). Esto facilitó la implementación del patrón de arquitectura Modelo-Vista-Controlador (MVC), separando de manera estricta el diseño visual de la lógica programada en las clases de Java.

2.4 Backend as a Service (BaaS) y Base de Datos: Supabase

Toda la infraestructura de backend, almacenamiento de datos y control de accesos fue delegada a Supabase, una alternativa de código abierto a Firebase.

Justificación técnica: Supabase se integró al proyecto para cumplir con tres funciones críticas:

- **Motor de Base de Datos:** Provee una base de datos relacional PostgreSQL alojada en la nube (AWS), a la cual el sistema se conecta mediante un Connection Pooler (puerto 6543) para ejecutar transacciones seguras.
- **Autenticación (SSO):** Su módulo de Auth gestiona el protocolo OAuth 2.0, permitiendo a los usuarios del CRM iniciar sesión utilizando proveedores de identidad externos como Google y GitHub de manera segura.
- **API REST Automatizada:** Genera instantáneamente endpoints (URLs) para interactuar con las tablas de la base de datos mediante peticiones HTTP estándar (GET, POST, PATCH, DELETE), característica que fue explotada directamente mediante JavaScript en el módulo web de la agenda

3. CONFIGURACIÓN DEL APLICATIVO

3.1 Configuración de API Keys y Servicios Externos

El aplicativo CRM-Tec se apoya en una arquitectura de microservicios e integra diversas plataformas en la nube para la gestión de datos, autenticación segura y obtención de recursos multimedia. Para que estos módulos operen de manera correcta en cualquier entorno de despliegue, es imperativo configurar las credenciales de acceso (API Keys y Client Secrets) en las clases correspondientes dentro del paquete APIs. A continuación, se detallan los servicios a configurar y los archivos asociados:

3.1.1 Supabase

Supabase funciona como el cerebro remoto del aplicativo. Además de la conexión JDBC de PostgreSQL, el sistema realiza peticiones HTTP a la API REST de Supabase para lectura de datos y control de accesos.

- Archivo a modificar: `src/APIs/Supabase.java` (y cualquier controlador que invoque servicios de OAuth).
- Procedimiento: Localice las variables estáticas y reemplace los valores por defecto:
 - `SUPABASE_URL`: Ingrese la URL base de su proyecto (ej. `https://[ID_PROYECTO].supabase.co`).
 - `SUPABASE_KEY`: Ingrese la clave pública anónima (anon public key) proporcionada en el panel de control del proyecto.

3.1.2 Google OAuth 2.0 y Firebase

Para ofrecer un inicio de sesión rápido y seguro, el CRM-Tec utiliza el flujo de Google Authorization Code y el Identity Toolkit de Firebase.

- Archivos a modificar: `src/APIs/OAuth.java` y `src/APIs/Firebase.java`.
- Procedimiento:

- En el archivo OAuth.java, asigne las credenciales obtenidas desde la Google Cloud Console a las variables CLIENT_ID y CLIENT_SECRET.
- Asegúrese de que el puerto configurado en el LocalServerReceiver (por defecto 8080) coincida con las URIs de redirección autorizadas en Google.
- En el archivo Firebase.java, asigne la clave web de su proyecto de Firebase a la variable WEB_API_KEY para permitir el intercambio de tokens de acceso.

3.1.3 API de Unsplash

Para enriquecer la experiencia de usuario y evitar interfaces vacías, el sistema consume la API de Unsplash, buscando y asignando automáticamente imágenes representativas a los productos registrados mediante el método buscarImagenDeProducto().

- Archivo a modificar: src/APIs/Unsplash_Service.java.
- Procedimiento: Reemplace el valor de la variable CLIENT_ID con la Access Key generada desde el portal de desarrolladores de Unsplash.

3.2 Configurar la base de datos

El aplicativo CRM-Tec utiliza PostgreSQL como motor de base de datos principal, alojado y gestionado a través de los servicios en la nube de Supabase. La conexión entre el aplicativo JavaFX y la base de datos se realiza mediante JDBC (Java Database Connectivity) a través de la clase Conexion.java ubicada en el paquete BaseDatos.

3.2.1 Requisitos Previos

Para que la conexión sea exitosa, el entorno de desarrollo y ejecución debe contar con el controlador JDBC de PostgreSQL (ej. postgresql-42.x.x.jar) agregado a las dependencias o al Build Path del proyecto.

3.2.2 Parámetros de Conexión

La configuración se realiza modificando las variables estáticas dentro del archivo Conexion.java. El aplicativo está configurado para utilizar el Connection Pooler de Supabase (puerto 6543), lo cual optimiza el rendimiento al manejar múltiples conexiones concurrentes.

Los parámetros que deben configurarse son:

- URL (Endpoint): jdbc:postgresql://aws-1-us-west-1.pooler.supabase.com:6543/postgres
- Usuario: El ID de usuario asignado por Supabase (ej. postgres.exzjdesnqcjdloebpwth).
- Contraseña: La contraseña asignada al rol de base de datos.

3.2.3 Procedimiento de Configuración

1. Abrir el archivo src/BaseDatos/Conexion.java en el entorno de desarrollo.
2. Localizar las siguientes líneas de código al inicio de la clase:

```
private static final String url = "jdbc:postgresql://aws-1-us-west-1.pooler.supabase.com:6543/postgres";  
private static final String user = "postgres.exzjdesnqcjdloebpwth";  
private static final String pass = "TyDollaCris";
```

3. Reemplazar los valores entre corchetes con las credenciales correspondientes al entorno (Desarrollo o Producción).
4. Guardar los cambios y recompilar el proyecto.

3.2.4 Validación de la Conexión

Para comprobar que la configuración fue exitosa, el aplicativo cuenta con un método de validación interno. Al ejecutar una acción que requiera base de datos (como el inicio de sesión), el método conectar() imprimirá en la consola del sistema:

- Éxito: "Conectaste bien la base viejo"
- Fallo: "Error no conecto la base de datos: [Detalle de la excepción SQL]"

CAPÍTULO 4. CASOS DE USO

En este capítulo se detallan las especificaciones de los casos de uso del sistema CRM-Tec, describiendo la interacción exacta entre el actor (Usuario/Sistema) y la aplicación para satisfacer los requerimientos funcionales previamente establecidos.

CU-01: Inicio de Sesión Local

Campo	Descripción
Descripción	Permite a un usuario existente acceder al sistema validando sus credenciales locales.
Actor	Usuario
Precondiciones	El sistema debe tener conexión activa a la base de datos PostgreSQL en Supabase. El usuario debe estar previamente registrado.
Flujo Principal	1. El usuario abre la aplicación y visualiza la pantalla de Inicio de Sesión (loginM.fxml). 2. El usuario ingresa su Correo Electrónico y Contraseña. 3. El usuario presiona el botón "Iniciar Sesión". 4. El sistema valida que los campos no estén vacíos. 5. El sistema consulta la base de datos buscando una coincidencia exacta de correo y contraseña. 6. El sistema extrae el id_persona y lo guarda en la sesión global. 7. El sistema redirige al usuario a la Pantalla Principal (principal.fxml).

Flujos Alternativos / Excepciones	A. Campos vacíos: Si el usuario omite el correo o contraseña, el sistema lanza la alerta "<i>Datos incompletos</i>" y detiene el proceso. B. Datos incorrectos: Si las credenciales no coinciden, lanza la alerta "<i>Datos incorrectos</i>" e incrementa el contador de intentos fallidos. C. Bloqueo temporal: Si el usuario falla 5 veces consecutivas, el sistema desactiva los controles de inicio de sesión durante 60 segundos por seguridad.
Postcondiciones	El usuario autenticado tiene acceso a los módulos principales del CRM correspondientes a su nivel de permisos.

CU-02: Inicio de Sesión con Plataformas Externas SSO

Campo	Descripción
Descripción	Permite acceder al sistema utilizando una cuenta de Google o GitHub mediante el protocolo OAuth administrado por Supabase.
Actor	Usuario
Precondiciones	El usuario debe contar con conexión a internet y el puerto local 8080 de su equipo debe estar libre.
Flujo Principal	1. El usuario hace clic en el botón de Google o GitHub en la pantalla de inicio. 2. El sistema abre el navegador web predeterminado con la URL de autorización correspondiente. 3. En paralelo, el sistema levanta un servidor local temporal (HttpServer) escuchando en el puerto 8080. 4. El usuario autoriza el acceso en la página web de Google/GitHub.

	5. El navegador es redirigido a http://localhost:8080/callback con el token de acceso. 6. El servidor local intercepta el token, extrae el correo electrónico asociado y lo busca en la base de datos. 7. El sistema confirma la existencia del usuario, detiene el servidor temporal y redirige a la Pantalla Principal.
Flujos Alternativos / Excepciones	A. Usuario Nuevo (Correo no encontrado): Si el correo extraído del token no existe en la base de datos, el sistema guarda el correo en memoria y redirige automáticamente a la pantalla de "Crear Cuenta" (Registro_usuario.fxml) para que complete su perfil.
Postcondiciones	La sesión del usuario queda activa en el aplicativo sin necesidad de haber ingresado una contraseña local.

CU-03: Creación de Usuario y Registro de Negocio

Campo	Descripción
Descripción	Permite a un usuario nuevo registrar sus datos personales, seleccionar su rol y configurar los datos iniciales de su empresa.
Actor	Usuario
Precondiciones	El correo electrónico a registrar no debe existir previamente en la base de datos.
Flujo Principal	1. El usuario selecciona la opción "Crear una" en la pantalla de inicio. 2. El sistema despliega el formulario de Registro de Usuario (Registro_usuario.fxml). 3. El usuario ingresa Nombre, Apellidos, Correo, Teléfono, Contraseña y selecciona el Rol ("Administrador"). 4. El sistema despliega

	dinámicamente los campos CURP y RFC, los cuales son llenados por el usuario. 5. El usuario presiona "Continuar". El sistema inserta los datos en las tablas usuarios y administrador, y redirige a la pantalla de Registro de Negocio. 6. El usuario ingresa el nombre de su negocio y los datos de dirección (Calle, Colonia, C.P., Municipio, Estado). 7. El usuario presiona "Finalizar". El sistema empaqueta la inserción usando una transacción SQL segura. 8. El sistema confirma el éxito y redirige a la Pantalla Principal.
Flujos Alternativos / Excepciones	A. Rol "Otro": Si el usuario selecciona un rol distinto a Administrador, los campos CURP y RFC permanecen ocultos y no son requeridos para avanzar. B. Error de Transacción: Si ocurre un error al guardar la dirección del negocio (ej. falla de red), el sistema ejecuta un rollback(), deshaciendo la creación del negocio para evitar datos inconsistentes, y muestra un mensaje de error.
Postcondiciones	El usuario queda registrado como administrador, con su negocio asociado activo y es dirigido al panel de control principal.

CU-04: Visualización de Inventario y Enriquecimiento Visual

Campo	Descripción
Descripción	Permite al usuario visualizar su catálogo de productos registrados en un formato de cuadrícula (Grid), mostrando el estado del stock y asignando automáticamente una imagen representativa obtenida de internet.
Actor	Usuario

Precondiciones	El usuario debe tener una sesión activa. El equipo debe contar con conexión a internet para consumir la API de Unsplash.
Flujo Principal	1. El usuario navega al módulo de Inventario a través del menú lateral. 2. El sistema inicia un hilo en segundo plano (new Thread) para evitar congelar la interfaz y ejecuta una consulta SQL a las tablas productos y stock. 3. El sistema extrae los datos de cada producto (nombre, caducidad, stock actual y capacidad máxima). 4. El sistema invoca al método <code>Unsplash_Service.buscarImagenDeProducto(nombre)</code> para buscar y descargar una fotografía relacionada con el nombre del producto. 5. El sistema empaqueta la información en el modelo <code>Producto</code> y utiliza <code>Platform.runLater()</code> para actualizar la interfaz de manera segura, renderizando las tarjetas dinámicas en el <code>GridPane (gridInventario)</code>.
Flujos Alternativos / Excepciones	A. Fallo de red o API: Si la conexión a la API de Unsplash falla, el sistema captura la excepción y puede omitir la carga de la imagen sin interrumpir la visualización de los datos numéricos del inventario.
Postcondiciones	El usuario obtiene una vista panorámica y gráfica de la disponibilidad de sus artículos.

CU-05: Registro de Nuevo Producto

Campo	Descripción
Descripción	Permite registrar un nuevo artículo en el sistema, definiendo sus detalles comerciales, su unidad de medida y configurando los límites de inventario para su control.
Actor	Usuario
Precondiciones	El usuario debe estar dentro de la pantalla de añadir producto en el módulo de Inventario.
Flujo Principal	1. El usuario llena los datos generales: Nombre del producto, Marca, Precio por Unidad y selecciona la Unidad de Medida en el ComboBox.

	2. El usuario define la configuración de almacenamiento: Contenido por envase, Stock Actual, Stock Mínimo y Stock Máximo. 3. De ser aplicable, el usuario selecciona una Fecha de Caducidad. 4. El usuario presiona el botón para guardar. 5. El sistema deshabilita el autocommit de la base de datos (conectar.setAutoCommit(false)) para iniciar una transacción atómica. 6. El sistema inserta los datos en la tabla productos y recupera el ID generado mediante la cláusula RETURNING id_producto. 7. Inmediatamente, el sistema utiliza ese ID para insertar la configuración de existencias en la tabla stock. 8. El sistema ejecuta el commit(), muestra una alerta de "<i>Registro exitoso</i>" y redirige al usuario a la vista principal del Inventario.
Flujos Alternativos / Excepciones	A. Error Transaccional: Si ocurre un error de SQL durante la inserción en la tabla stock (ej. un tipo de dato incorrecto), el sistema atrapa la excepción y ejecuta un conectar.rollback(), cancelando todo el proceso para evitar que un producto quede registrado sin su respectivo control de inventario.
Postcondiciones	El producto es añadido a la base de datos y aparecerá automáticamente al recargar la cuadrícula de inventario.

CU-06: Gestión de Agenda y Calendario Interactivo

Campo	Descripción
Descripción	Permite al usuario visualizar, programar, modificar (arrastrar y soltar) y eliminar eventos en un

	calendario interactivo que se sincroniza en tiempo real con la base de datos.
Actor	Usuario
Precondiciones	El usuario debe tener una sesión activa en el sistema. Es obligatoria la conexión a internet para cargar las librerías de <i>FullCalendar</i> y realizar las peticiones a la API REST de Supabase.
Flujo Principal	1. El usuario navega al módulo de Agenda. 2. El sistema inicializa un componente WebView y un WebEngine que cargan el recurso local calendar.html. 3. Al confirmar la carga exitosa (State.SUCCEEDED), el controlador JavaFX inyecta el ID del usuario en sesión ejecutando el script de JavaScript inicializarCalendario(id). 4. El entorno web realiza una petición GET a Supabase para obtener y renderizar los eventos exclusivos de ese usuario. 5. Crear Evento: El usuario hace clic en una fecha vacía. El WebEngine intercepta el prompt de JavaScript y lanza un TextInputDialog nativo de JavaFX. El usuario escribe el título, acepta, y el sistema realiza un POST a Supabase para guardarlo. 6. Modificar Evento: El usuario arrastra un evento a otro día (eventDrop). El entorno web captura la nueva fecha y envía una petición PATCH a Supabase actualizando el registro instantáneamente. 7. Eliminar Evento: El usuario hace clic sobre un evento existente. El controlador intercepta el confirm de JavaScript desplegando una alerta nativa de JavaFX. Al confirmar, se

	envía una petición DELETE a Supabase y el evento desaparece de la vista.
Flujos Alternativos / Excepciones	A. Error de Red: Si falla cualquier petición a la base de datos (crear, mover o eliminar), el bloque catch de JavaScript lanza una alerta web que es interceptada por el controlador (setOnAlert) para mostrar al usuario un mensaje de error estilo JavaFX, revirtiendo visualmente el cambio en el calendario (info.revert()).
Postcondiciones	La tabla eventos en la base de datos refleja con exactitud la programación del usuario y los cambios son visibles de inmediato.

CU-07: Visualización, Búsqueda y Gestión de Estado de Tareas

Campo	Descripción
Descripción	Permite al usuario visualizar su lista de tareas asignadas, buscar actividades específicas y marcar tareas como completadas directamente desde la interfaz.
Actor	Usuario
Precondiciones	El usuario debe tener una sesión activa. La base de datos debe estar conectada.
Flujo Principal	1. El usuario ingresa al módulo presionando el botón de Tareas en el menú lateral. 2. El sistema carga la vista y ejecuta una consulta a la tabla tareas, filtrando únicamente los registros donde el id_usuario corresponda a la sesión activa (SesionGlobal). 3. El sistema estructura la información obtenida utilizando el modelo Tarea.java (Nombre, Fecha, Horarios, Descripción, Prioridad, Responsable y Estado) y la despliega en un TableView. 4. Búsqueda Dinámica: El usuario

	escribe en la barra de búsqueda (txtBuscarTarea). El sistema filtra los resultados en tiempo real usando un FilteredList, mostrando solo las tareas que coincidan en actividad, descripción, prioridad o responsable. 5. Actualización de Estado: El usuario interactúa con la columna de estado marcando el CheckBox de una fila. 6. El sistema detecta el cambio de estado (isCompletada()) y ejecuta un UPDATE tareas SET completada = ? WHERE id_tarea = ? en la base de datos de manera asíncrona.
Flujos Alternativos / Excepciones	A. Tareas no encontradas: Si el usuario no tiene tareas registradas o la búsqueda no coincide con ninguna, la tabla se mostrará vacía sin interrumpir el flujo.
Postcondiciones	El usuario puede llevar un control visual exacto de sus pendientes y el sistema mantiene actualizada la tabla tareas con los estados reales.
CU-08: Registro y Asignación de Nueva Tarea	
Campo	Descripción
Descripción	Permite registrar una nueva actividad operativa o administrativa en el sistema, definiendo sus horarios, nivel de prioridad y el responsable a cargo.
Actor	Usuario
Precondiciones	El usuario debe encontrarse en el módulo principal de Tareas.
Flujo Principal	1. El usuario presiona el botón para agregar una nueva tarea. 2. El sistema carga el formulario de registro (agrtarea.fxml o similar) a través de Controller_RTarea.java. 3. El usuario ingresa la información requerida: Nombre de la actividad, Horario de inicio, Horario de fin,

	Descripción y el nombre del Responsable. 4. El usuario selecciona la fecha límite mediante el DatePicker (dtFecha). 5. El usuario selecciona la prioridad utilizando los RadioButton (Alta, Media o Baja). 6. El usuario presiona el botón de "Aceptar" (btnAceptar). 7. El sistema valida los datos y prepara una inserción (INSERT INTO tareas) vinculando el registro al id_usuario actual. 8. El sistema muestra la alerta de éxito <i>"La tarea se registro exitosamente"</i> y redirige al usuario de vuelta a la vista principal de Tareas.
Flujos Alternativos / Excepciones	A. Fecha no especificada: Si el usuario decide no seleccionar una fecha en el DatePicker, el sistema atrapa esta condición y almacena un valor nulo seguro (java.sql.Types.DATE) en la base de datos para evitar errores de tipo NullPointerException. B. Cancelación: Si el usuario presiona "Cancelar", el sistema descarta los datos introducidos y devuelve al usuario a la vista de tareas sin realizar alteraciones en la base de datos.
Postcondiciones	La nueva tarea aparece listada en el panel de Tareas y su estado se inicializa por defecto como "no completada".

CU-09: Visualización, Búsqueda y Filtrado de Contactos

Campo	Descripción
Descripción	Permite al usuario visualizar su directorio de contactos, aplicar filtros por categoría de rol y realizar búsquedas dinámicas en tiempo real.

Actor	Usuario
Precondiciones	El usuario debe tener una sesión activa. Debe existir al menos un registro en la base de datos asociado al id_propietario del usuario actual.
Flujo Principal	1. El usuario navega al módulo haciendo clic en el botón de Contactos. 2. El sistema ejecuta el método cargarContactosBD(), consultando la tabla usuarios mediante un JOIN con ocupacion, filtrando estrictamente los registros donde id_propietario coincida con la sesión activa y que contrasena IS NULL (para no mezclar administradores con contactos). 3. El sistema formatea los datos mediante el modelo Contacto.java y puebla la tabla mostrando Nombre Completo, Número Telefónico y Ocupación. 4. Búsqueda (RF9): El usuario ingresa texto en el campo de búsqueda (txtfBusContacto). El sistema escucha los cambios (textProperty().addListener) y actualiza instantáneamente la tabla si el texto coincide con el nombre completo o el número telefónico del contacto. 5. Filtrado (RF9): El usuario presiona uno de los botones categóricos (Empleados, Proveedores, Clientes, Desarrolladores). El sistema aplica un Predicate a la lista filtrada (listaFiltrada) mostrando únicamente las coincidencias exactas con esa ocupación.
Flujos Alternativos / Excepciones	A. Directorio Vacío: Si la consulta a la base de datos no devuelve resultados (o si el filtro/búsqueda no encuentra coincidencias), la tabla de la interfaz simplemente se mostrará en blanco sin interrumpir el funcionamiento del sistema.

Postcondiciones	El usuario visualiza la información segmentada y tiene habilitada la opción para agregar nuevos registros.
------------------------	--

CU-10: Registro de Nuevo Contacto

Campo	Descripción
Descripción	Permite agregar un nuevo registro al directorio del negocio asignándole una ocupación específica.
Actor	Usuario
Precondiciones	El usuario debe encontrarse en el módulo de Contactos.
Flujo Principal	1. En la pantalla de Contactos, el usuario presiona el botón "Agregar Contacto" (btnAgrCont). 2. El sistema navega hacia la vista del formulario (agrContacto.fxml). 3. El usuario llena los campos obligatorios: Nombre, Apellido Paterno, Apellido Materno, Correo, Número Telefónico, y selecciona la Ocupación desde el menú desplegable (cmbboxOcupacion). 4. El usuario presiona "Guardar". 5. El sistema (Controller_AContacto.java) valida que ningún campo de texto esté vacío y que se haya seleccionado un ítem del menú. 6. El sistema traduce la selección de texto a un identificador numérico (idOcupacion del 1 al 6 mediante un bloque switch). 7. El sistema ejecuta un INSERT en la tabla usuarios, vinculando el registro al id_propietario (ID del administrador en sesión). 8. El sistema muestra la alerta "Contacto guardado exitosamente" y

	limpia todos los campos de texto para permitir un nuevo registro.
Flujos Alternativos / Excepciones	A. Campos vacíos (RF10): Si falta algún dato, el sistema bloquea el registro y lanza la advertencia <i>"Completa todos los datos del contacto"</i>. B. Formato de número inválido: Si el usuario ingresa caracteres alfabéticos en el campo de número telefónico, el código atrapa la excepción NumberFormatException y muestra el error <i>"No puedes colocar letras en el campo de número de telefono"</i>.
Postcondiciones	El nuevo contacto es añadido a la base de datos y será visible en la tabla general al volver a cargar el módulo principal de contactos.

CU-11: Interacción con el Asistente Inteligente

Campo	Descripción
Descripción	Permite al usuario realizar consultas, pedir consejos o redactar textos utilizando un asistente conversacional integrado directamente en el CRM, impulsado por el modelo Gemini 1.5 Flash.
Actor	Usuario
Precondiciones	Conexión a internet estable y acceso a la clave de la API de Google Generative AI.
Flujo Principal	1. El usuario navega al módulo de IA. 2. El usuario escribe una instrucción o pregunta en el cuadro de texto (txtMensaje) y presiona "Enviar" (btnEnviar). 3. El sistema extrae el texto, limpia el campo de entrada y renderiza una burbuja de chat en la interfaz alineada a la derecha para representar el mensaje del usuario.

	4. El sistema actualiza el historial de conversación en formato JSON para mantener el contexto. 5. El sistema construye una petición HTTP asíncrona (CompletableFuture.runAsync) tipo POST dirigida al endpoint de la API de Gemini, incluyendo el historial y el nuevo mensaje. 6. Al recibir una respuesta HTTP 200, el sistema procesa el JSON resultante, extrae el bloque de texto generado por la IA y renderiza una nueva burbuja de chat alineada a la izquierda. 7. El sistema actualiza nuevamente el historial guardando la respuesta del modelo.
Flujos Alternativos / Excepciones	A. Error de API: Si la API devuelve un código de error (ej. sobrecarga del servidor o error de autenticación), el sistema muestra una burbuja de chat indicando <i>"Error de API: [código]"</i>. B. Falla de conexión: Si ocurre una excepción de red durante la petición HTTP, el sistema atrapa el error y muestra una alerta visual en el chat indicando <i>"Error de conexión"</i>.
Postcondiciones	El usuario obtiene una respuesta generada por inteligencia artificial sin necesidad de abandonar el entorno del sistema CRM.

CAPÍTULO 5. MÓDULO DE ADMINISTRACIÓN

El Módulo de Administración en CRM-Tec comprende el conjunto de herramientas, configuraciones y privilegios exclusivos para el usuario catalogado como "dueño" o "administrador" del sistema. Este módulo garantiza que la información sensible de la empresa y la asignación de roles se manejen de forma segura y centralizada.

5.1 Gestión de Roles y Privilegios

El sistema maneja un control de acceso basado en roles (RBAC - Role-Based Access Control), el cual se define desde el momento de la creación de la cuenta a través del controlador `Controller_RUsuario.java`.

- **Asignación de Nivel de Acceso:** Cuando un usuario se registra, debe seleccionar su rol mediante el componente `cmbOcupacion`. El sistema traduce esta selección a un identificador numérico (`idOcupacion`).
- **Privilegios de Administrador (Rol 1):** Si el sistema detecta que el `idOcupacion` es igual a 1 (Administrador), se activa un flujo de validación más estricto. El aplicativo requiere obligatoriamente que el usuario ingrese su Clave Única de Registro de Población (CURP) y su Registro Federal de Contribuyentes (RFC).
- **Integridad Referencial:** Al guardar, el sistema no solo crea el perfil en la tabla base usuarios, sino que ejecuta una inserción paralela en la tabla administrador, vinculando ambos registros mediante el `id_persona` generado. Esto le otorga a esa cuenta los permisos máximos dentro del CRM.

5.2 Configuración del Perfil Empresarial (Negocio)

Una función exclusiva del perfil de administrador es la configuración y modificación de los datos comerciales y logísticos de la empresa, gestionada a través del controlador `Controller_RNegocio.java`.

- **Registro de Entidad Comercial:** El administrador es el único facultado para registrar el nombre oficial del emprendimiento (`txtNombre_negocio`).

- Configuración de Domicilio: Se requiere capturar la matriz de dirección de la empresa, abarcando: Calle, Colonia, Código Postal, Municipio y Estado.
- Seguridad Transaccional de Configuración: Debido a la importancia de estos datos, el proceso de guardado desactiva el AutoCommit de la base de datos. El sistema agrupa la inserción del negocio y su dirección en una transacción atómica; si ocurre un error (por ejemplo, ingresar letras en el campo de Código Postal, lo cual lanza un `NumberFormatException`), el sistema ejecuta un `rollback()` para evitar configuraciones empresariales corruptas o a medias.

5.3 Segmentación de Espacios de Trabajo (Data Isolation)

Todas las operaciones posteriores que ocurren en el sistema (como la agenda de clientes o la asignación de tareas) nacen de este módulo de administración. El ID del administrador activo (`id_persona`) se almacena en memoria global y se utiliza como una llave de propiedad (`id_propietario` o vinculaciones directas a tareas/contactos). Esto significa que el administrador actúa como la raíz del árbol, y los datos generados por sus empleados o para sus clientes están tecnológicamente aislados de otras empresas que utilicen el mismo aplicativo.

CAPITULO 6. Modelo Entidad-Relación

Este capítulo documenta la vista total del modelo entidad-relación desarrollado para la creación de la base de datos de el sistema divididos en 4 imagenes con sus correspondientes campos en sus respectivas tablas.

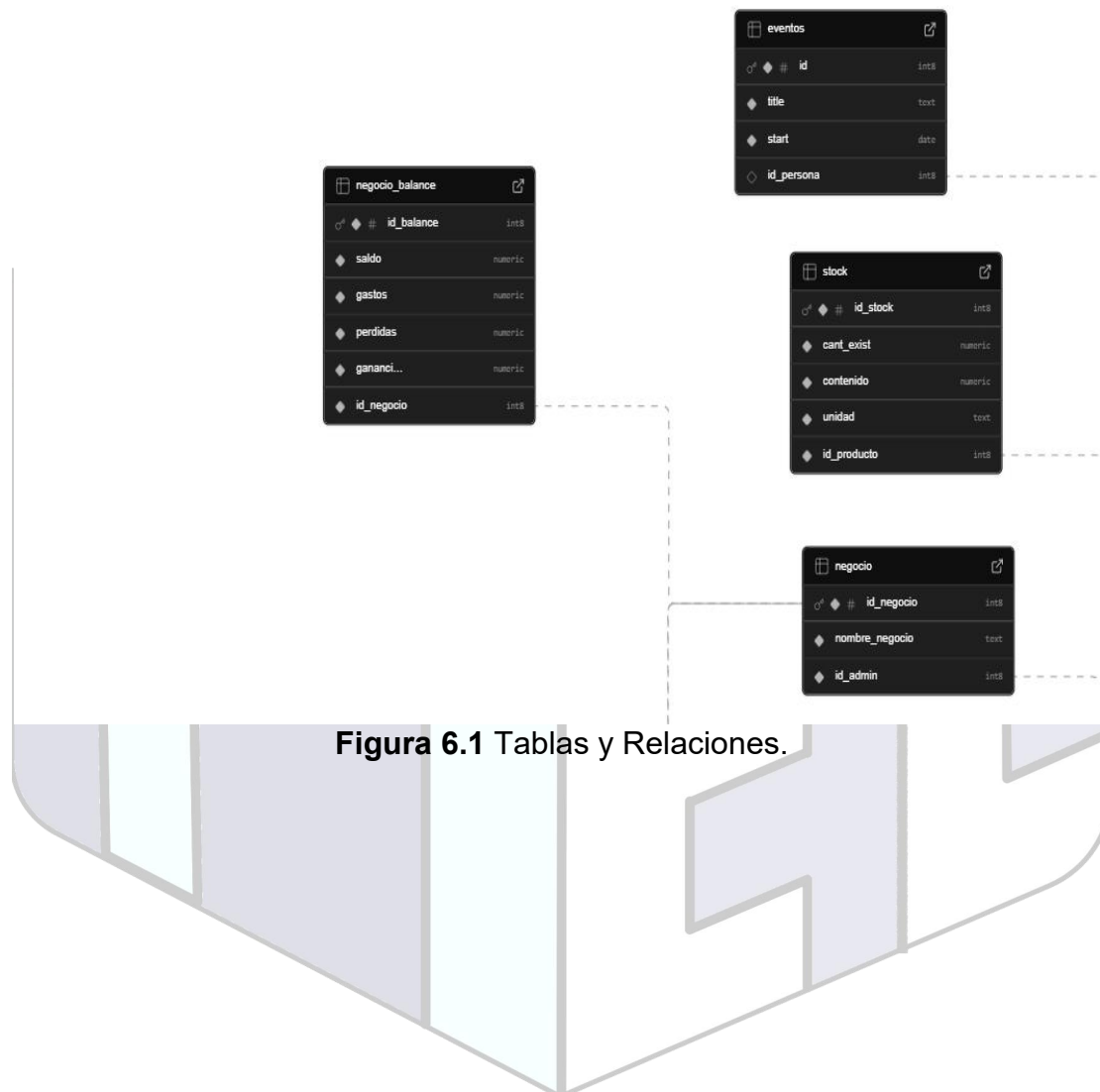


Figura 6.1 Tablas y Relaciones.

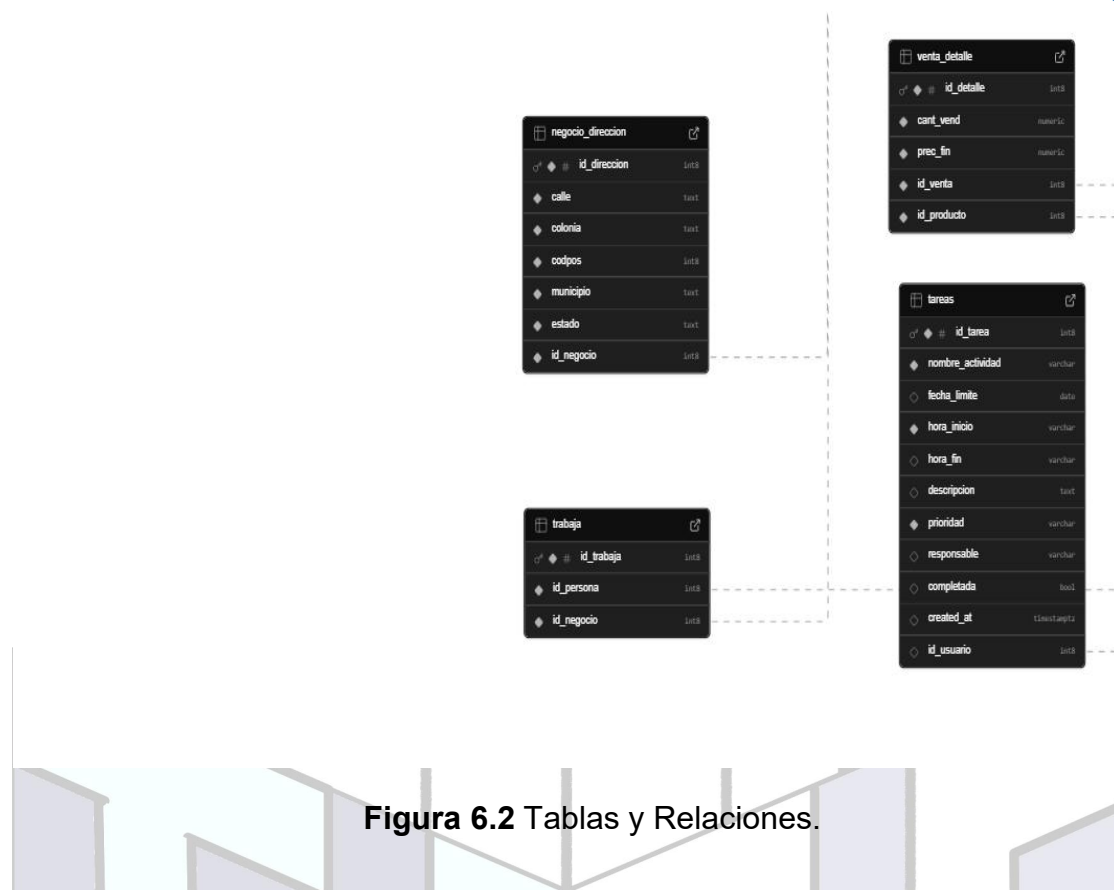


Figura 6.2 Tablas y Relaciones.



Figura 6.3 Tablas y Relaciones.



Figura 6.4 Tablas y Relaciones.

CAPITULO 7 DICCIONARIO DE DATOS DEL MODELO ENTIDAD-RELACIÓN

Este capítulo documenta el diccionario de datos redactado a travez de una serie de tablas que tratan con las tablas correspondientes al modelo entidad-relación de la base de datos diseñada, junto a ellas viene el titulo de a que apartado corresponden.

Ocupación

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_ocupacion (PK)	bigint(8)	No				
nombre_ocupacion	Text	No				

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	Id_ocupacion	0	N/A	No

Productos

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_producto (PK)	bigint(8)	No				
nombre_prod	text	No				
marca	text	No				
caducidad	date	Si				
precio	double	No				
fecha_recepcion	date	Si				
id_persona (FK)	bigint(8)	Si				

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_producto	0	N/A	No

Usuarios

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_persona (PK)	bigint(8)	No				
nombre	text	No				
ap_pat	text	No				
ap_mat	Text	No				
correo	Text	No				
num_tel	numeric	No				
contrasena	text	Si				
id_propietario	bigint(8)	Si				
id_ocupacion (FK)	bigint(8)	No		Ocupacion -> id_ocupacion		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_persona	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_ocupacion	0	N/A	No
FOREIGN	BTREE	No	N/A	id_propietario	0	N/A	Si

Stock

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_stock (PK)	bigint(8)	No				
cant_exist	Numeric	No				
contenido	Numeric	Si				
unidad	text	Si				
id_producto (FK)	numeric	No		Productos -> id_producto		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_stock	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_producto	0	N/A	No

Administrador

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_admin (PK)	bigint(8)	No				
curp	varchar	No				
rfc	varchar	No				
id_persona (FK)	bigint(8)	No		usuarios -> id_persona		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_admin	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_persona	0	N/A	No

Ventas

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_venta (PK)	bigint(8)	No				
Fecha_venta	Date	No				
id_persona (FK)	bigint(8)	No		usuarios -> id_persona		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_venta	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_persona	0	N/A	No

Provee

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
---------	------	------	----------------	---------	-------------	------

id_provee (PK)	bigint(8)	No				
Fecha_entrega	Date	No				
Cant_com	numeric	No				
Id_producto (FK)	bigint(8)	No		Productos -> id_producto		
id_persona (FK)	bigint(8)	No		usuarios -> id_persona		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_provee	0	0	No
FOREIGN	BTREE	No	N/A	Id_producto	0	0	No
FOREIGN	BTREE	No	N/A	Id_persona	0	0	No

Negocio

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_negocio (PK)	bigint(8)	No				
Nombre_negocio	text	No				
id_admin (FK)	bigint(8)	No		administrador -> id_admin		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_provee	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_admin	0	N/A	No

Negocio_balance

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_balance (PK)	bigint(8)	No				
Saldo	numeric	No				
Gastos	numeric	No				
perdidas	Numeric	No				
ganancias	numeric	No				
id_negocio (FK)	bigint(8)	No		Negocio -> id_negocio		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_balance	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_negocio	0	N/A	No

Negocio_direccion

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_direccion (PK)	bigint(8)	No				
calle	Text	No				
colonia	text	No				
codpos	Bigint(8)	No				
municipio	text	No				
estado	Text	No				
id_negocio (FK)	bigint(8)	No		Negocio → id_negocio		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_direccion	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_negocio	0	N/A	No

trabaja

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_trabaja (PK)	bigint(8)	No				

Id_persona (PK)	Bigint(8)	No		usuarios → id_persona		
id_negocio (FK)	bigint(8)	No		Negocio → id_negocio		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_trabaja	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_persona	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_negocio	0	N/A	No

Venta_detalle

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_detalle (PK)	bigint(8)	No				
Cant_vend	numeric	No				
Prec_fin	numeric	No				
Id_venta (FK)	bigint(8)	No		Ventas → id_venta		
id_producto (FK)	bigint(8)	No		productos → id_producto		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_detalle	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_venta	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_producto	0	N/A	No

tareas

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_tarea (PK)	bigint(8)	No				
nombre_actividad	varchar	No				
fecha_limite	date	Si				
hora_inicio	varchar	No				
hora_fin	varchar	Si				
descripcion	text	Si				
prioridad	varchar	No				
responsable	varchar	No				
completada	boolean	Si				
created_at	timestampz	Si				
id_usuario(FK)	bigint(8)	No		Usuarios -> id_persona		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_tarea	0	N/A	No
FOREIGN	BTREE	No	N/A	id_usuario	0	N/A	No

eventos

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id(PK)	bigint(8)	No				
title	varchar	No				
start	date	Si				
id_persona(FK)	varchar	No		Usuarios -> id_persona		

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id	0	N/A	No
FOREIGN	BTREE	No	N/A	id_persona	0	N/A	No

CAPITULO 8. PROTOTIPOS DE PANTALLAS DEL APLICATIVO

Este capítulo documenta las interfaces gráficas de usuario (GUI) que componen el aplicativo CRM-Tec, detallando su propósito, su composición visual y el funcionamiento de su controlador asociado.

8.1 Pantalla de Inicio de Sesión (loginM.fxml)

Función principal: Permitir el acceso seguro al sistema validando la identidad del usuario. Ofrece dos vías de autenticación: mediante credenciales locales (correo y contraseña) o a través de Single Sign-On (SSO) utilizando cuentas de Google o GitHub.

Diseño de la interfaz (loginM.fxml):

La estructura visual está maquettata mediante un GridPane que divide la ventana en dos secciones principales. Los componentes gráficos clave incluyen:

- Campos de entrada de datos (TextField y PasswordField) para capturar el correo y la contraseña.
- Un botón principal estándar para ejecutar el inicio de sesión local.
- Botones iconográficos interactivos para accionar el inicio de sesión con proveedores externos (Google y GitHub).
- Una etiqueta de texto (Label) que funciona como hipervínculo para redirigir a los usuarios sin cuenta hacia el formulario de registro.

Funcionamiento del Controlador (Controller.java):

El controlador gestiona los eventos de la vista y aplica la lógica de seguridad y acceso:

- Validación Local: A través del método `verificacion()`, el controlador captura los datos de los campos de texto y realiza una consulta SQL a la

base de datos. Si coinciden, guarda el ID del usuario en sesión y renderiza la pantalla principal.

- **Prevención de Fuerza Bruta:** Cuenta con un método de seguridad que bloquea la interfaz durante un minuto si detecta 5 intentos fallidos de inicio de sesión consecutivos.
- **Gestión SSO:** Al solicitar acceso vía Google o GitHub, el controlador abre el navegador predeterminado y levanta un servidor local temporal (puerto 8080) para interceptar el token de acceso devuelto por Supabase, validando la identidad del usuario en segundo plano y otorgando el acceso.

```
package Controladores;

public class Contacto {

    private String nombreCompleto;
    private String numeroTelefono;
    private String ocupacion;

    public Contacto (String nombreCompleto, String numeroTelefono, String ocupacion){
        this.nombreCompleto = nombreCompleto;
        this.numeroTelefono = numeroTelefono;
        this.ocupacion = ocupacion;
    }

    public String getNombreCompleto(){
        return nombreCompleto;
    }

    public String getNumeroTelefono(){
        return numeroTelefono;
    }

    public String getOcupacion(){
        return ocupacion;
    }

}
```

8.2 Pantalla de Registro de Usuario

Función principal: Permitir la creación de nuevas cuentas en el sistema, capturando la información personal del usuario y asignándole un nivel de acceso (rol). Si el usuario se registra como "Administrador", el sistema exige credenciales de identificación oficial.

Diseño de la interfaz (Registro_usuario.fxml):

La vista está construida sobre un GridPane que aloja un ScrollPane para garantizar la correcta visualización de todos los campos en pantallas más pequeñas. Los componentes gráficos incluyen:

- Campos de texto (TextField y PasswordField) para capturar Nombre, Apellidos, Correo, Teléfono y Contraseña.
- Un menú desplegable (ComboBox) para seleccionar la ocupación o rol en el sistema.
- Campos condicionales para capturar el CURP y el RFC.
- Un panel lateral decorativo (VBox) con la clase de estilo menu-lateral.

Funcionamiento del Controlador (Controller_RUsuario.java):

- Inserción Base: Recupera los datos de los campos de texto e inserta el nuevo registro en la tabla usuarios.
- Gestión de Roles: Verifica el identificador numérico de la ocupación seleccionada. Si el rol equivale a Administrador (idOcupacion == 1), el controlador toma el id_persona recién generado y ejecuta una segunda consulta SQL para registrar los datos fiscales en la tabla administrador.
- Persistencia de Sesión: Guarda el ID del usuario en la clase estática SesionGlobal y redirige a la pantalla de configuración del negocio.

```
public class Controller_RUsuario {  
    public void alerta(String titulo, String mensaje, AlertType tipo) {  
        Alert alerta = new Alert(tipo);  
        alerta.setTitle(titulo);  
        alerta.setContentText(mensaje);  
        alerta.showAndWait();  
    }  
  
    public void verificacion() {  
        if (txtEmail.getText().isEmpty() || txtNombre.getText().isEmpty() ||  
            txtAp_mat.getText().isEmpty() || txtAp_pat.getText().isEmpty() ||  
            txtNumTel.getText().isEmpty() || txtContraseña.getText().isEmpty() || cmbOcupacion.getValue() == null) {  
            alerta(titulo: "Datos incompletos", mensaje: "Por favor, complete todos los campos", AlertType.WARNING);  
            return;  
        }  
        if ("Administrador".equals(cmbOcupacion.getValue())) {  
            if (txtCURP.getText().isEmpty() || txtRFC.getText().isEmpty()) {  
                alerta(titulo: "Datos incompletos", mensaje: "Si eres admin ingresa curp y RFC", AlertType.WARNING);  
                return;  
            }  
        }  
        guardarCrisSupaboy();  
    }  
  
    private void guardarCrisSupaboy() {  
        Connection conectable = Conexion.conectar();  
        if (conectable == null) {  
            alerta(titulo: "Error de conexión", mensaje: "No se conecto a la base viejon", AlertType.ERROR);  
            return;  
        }  
        // Esto es para saber si eligio admin = 1 o otro = 2  
        String ocupacion = cmbOcupacion.getValue();  
        long idOcupacion = "Administrador".equals(ocupacion) ? 1 : 2;  
  
        String sqlUsuario = "INSERT INTO usuarios (nombre, ap_pat, ap_mat, correo, num_tel, id_ocupacion, contraseña) VALUES (?, ?, ?, ?, ?, ?) RETURNING id_persona";  
        String sqlAdmin = "INSERT INTO administrador (curp, rfc, id_persona) VALUES (?, ?, ?)";  
  
        try {
```

8.3 Pantalla de Registro de Negocio

Función principal: Capturar y vincular la información comercial, logística y de ubicación del emprendimiento al perfil del administrador que acaba de crear su cuenta.

Diseño de la interfaz (registrar_negocio.fxml):

Se estructura mediante un GridPane simétrico con las siguientes características:

- Formularios (TextField) etiquetados para el ingreso del nombre comercial del negocio y su dirección detallada: Calle, Colonia, Código Postal, Municipio y Estado.
- Una barra inferior (HBox) que contiene los botones de acción "Regresar" (para cancelar el proceso) y "Continuar" (para guardar los datos).

Funcionamiento del Controlador (Controller_RNegocio.java):

- Seguridad Transaccional: Al presionar "Continuar", el controlador desactiva el autoguardado de la base de datos (`conectar.setAutoCommit(false)`).
- Inserción Relacional: Inserta primero el nombre en la tabla negocio, recupera el ID generado, y lo utiliza como llave foránea para insertar los datos geográficos en la tabla negocio_direccion.
- Manejo de Excepciones: Si el usuario ingresa texto en el campo numérico del Código Postal, se captura la excepción `NumberFormatException` y el controlador ejecuta un `rollback()` para deshacer cualquier cambio parcial en la base de datos. Si todo es correcto, confirma la transacción (`commit()`) y redirige al dashboard principal.

```

public class Controller_RNegocio {
    private void guardarNegocioEnBase() {
        long idPersona = SesionGlobal.getIdUsuarioActivo();
        System.out.println("ID en la mochila buscando permisos de Admin: " + idPersona);
        if (idPersona == 0) {
            alerta(titulo: "Error", mensaje: "No hay un usuario activo, por favor vuelve a iniciar sesion", AlertType.ERROR);
        }
        Connection conectar = Conexion.conectar();
        if (conectar == null)
            return;

        String sqlConsultaAdmin = "SELECT id_admin FROM administrador WHERE id_persona=?";
        String sqlInsertNegocio = "INSERT INTO negocio (nombre_negocio, id_admin) VALUES (?,?)";
        String sqlInsertDireccionNeg = "INSERT INTO negocio_direccion (calle, colonia, codpos, municipio, estado, id_negocio) VALUES (?, ?, ?, ?, ?, ?)";

        try {
            PreparedStatement psAdmin = conectar.prepareStatement(sqlConsultaAdmin);
            psAdmin.setLong(parameterIndex: 1, idPersona);
            ResultSet rsAdmin = psAdmin.executeQuery();

            long idAdmin = 0;
            if (rsAdmin.next()) {
                idAdmin = rsAdmin.getLong(columnLabel: "id_admin");
            } else {
                alerta(titulo: "Error", mensaje: "Acceso denegado. Tu cuenta no tiene permisos de administrador", AlertType.ERROR);
                return;
            }

            conectar.setAutoCommit(autoCommit: false);
            PreparedStatement psInsertarNegocio = conectar.prepareStatement(sqlInsertNegocio,
                java.sql.Statement.RETURN_GENERATED_KEYS);
            psInsertarNegocio.setString(parameterIndex: 1, txtNombre_negocio.getText());
            psInsertarNegocio.setLong(parameterIndex: 2, idAdmin);
            psInsertarNegocio.executeUpdate();

            ResultSet rsInsertarNegocio = psInsertarNegocio.getGeneratedKeys();
            if (rsInsertarNegocio.next()) {
                long idNegocioGenerado = rsInsertarNegocio.getLong(columnIndex: 1);
                PreparedStatement psInsertDireccion = conectar.prepareStatement(sqlInsertDireccionNeg);
            }
        }
    }
}

```

8.4 Pantalla Principal (Dashboard)

Función principal: Servir como el panel de control central (hub) del CRM-Tec. Presenta un resumen visual del estado general de la empresa, mostrando estadísticas de inventario, tareas pendientes y accesos rápidos a los contactos recientes.

Diseño de la interfaz (principal.fxml):

Es la vista más robusta del aplicativo y está dividida en dos grandes áreas:

- Menú Lateral (Sidebar): Un VBox que actúa como barra de navegación global, conteniendo botones de tipo ToggleButton con íconos para cada submódulo (Agenda, Inventario, Tareas, IA, etc.).
- Área de Trabajo: Contenedores tipo tarjeta (AnchorPane / VBox) que muestran las métricas rápidas. Además, integra un componente TableView configurado para listar los últimos registros del directorio.

Funcionamiento del Controlador (Controller_Principal.java):

- Renderizado de Métricas: Al inicializar, el controlador ejecuta hilos y consultas SQL agrupadas para calcular cuántas tareas están terminadas y cuántas pendientes.
- Monitoreo de Stock: Consulta el inventario del usuario y, si detecta productos con cantidad menor o igual a cero, actualiza la interfaz visual para advertir un estado de "Desabastecido".
- Población de Tablas: Ejecuta un JOIN entre las tablas usuarios y ocupacion filtrando por el propietario actual, limitando el resultado a 4 registros para llenar dinámicamente el TableView de "Últimos Contactos".
- Navegación: Gestiona los eventos de clic de los botones del menú lateral, utilizando una clase utilitaria (Navegador) para inyectar nuevas vistas (.fxml) en la pantalla actual sin necesidad de abrir múltiples ventanas.

```
public class Controller_Principal{  
    public void initialize(){  
        btnInventario.setOnAction(event ->{  
            Navegador.Carga(btnInventario, vista: "/Vistas/Inventario.fxml");  
        });  
        btnAgenda.setOnAction(event ->{  
            Navegador.Carga(btnAgenda, vista: "/Vistas/Agenda.fxml");  
        });  
        btnTareas.setOnAction(event ->{  
            Navegador.Carga(btnTareas, vista: "/Vistas/Tareas.fxml");  
        });  
        btnContactos.setOnAction(event ->{  
            Navegador.Carga(btnContactos, vista: "/Vistas/contacto.fxml");  
        });  
        btnGraficas.setOnAction(event ->{  
            Navegador.Carga(btnGraficas, vista: "/Vistas/Graficas.fxml");  
        });  
        btnIA.setOnAction(event ->{  
            Navegador.Carga(btnIA, vista: "/Vistas/IA.fxml");  
        });  
        btnGuiaUsuario.setOnAction(event ->{  
            //Carga(btnGuiaUsuario, "/Vistas/");  
        });  
        btnSalir.setOnAction(event ->{  
            Alert alerta = new Alert(AlertType.CONFIRMATION);  
            alerta.setTitle("¿Estas seguro de salir?");  
            alerta.setContentText("¿Estas seguro de salir?");  
            Optional<ButtonType> result = alerta.showAndWait();  
            if (result.isPresent() && result.get() == ButtonType.OK) {  
                Navegador.Carga(btnSalir, vista: "/Vistas/loginM.fxml");  
            }  
        });  
        actualizarEstadisticasTareas();  
        actualizarEstadisticasInventario();  
        cargarCalendarioWebDiario();  
        actualizarAgendaPrincipal();  
    }  
}
```

8.5 Pantalla de Inventario

Función principal: Mostrar el catálogo de productos registrados en el sistema, permitiendo al usuario visualizar rápidamente la disponibilidad de stock y acceder al formulario para añadir nuevos artículos. Se enriquece visualmente consumiendo la API de Unsplash para ilustrar cada producto.

Diseño de la interfaz (Inventario.fxml):

La estructura base utiliza un contenedor HBox que separa el menú lateral del área de contenido principal. Sus componentes destacados son:

- Un área de visualización basada en un GridPane (gridInventario) envuelto en un ScrollPane para permitir el desplazamiento por el catálogo.
- Un botón principal ("Siguiete →" o "Añadir") para redirigir a la vista de registro de producto.
- Una etiqueta informativa en la parte inferior ("Powered by Unsplash API") para dar crédito al servicio externo de imágenes.

Funcionamiento del Controlador (InventarioController.java):

- Carga Asíncrona: Ejecuta un hilo en segundo plano (new Thread) para consultar la tabla productos y stock en la base de datos sin congelar la interfaz visual.
- Enriquecimiento Visual: Por cada producto recuperado, invoca al servicio Unsplash_Service para descargar una imagen relacionada con el nombre del producto.
- Renderizado Seguro: Utiliza Platform.runLater() para inyectar dinámicamente las tarjetas de los productos (con su foto, nombre y cantidades) dentro del gridInventario.


```

public class InventarioController {

    public void cargarDatosDesdeBD() {
        long idUsuarioActivo = SesionGlobal.getIdUsuarioActivo();
        new Thread(() -> {
            List<Producto> listaProductos = new ArrayList<>();
            Connection conectar = null;
            PreparedStatement ps = null;
            ResultSet rs = null;

            try {
                conectar = BaseDatos.Conexion.conectar();

                String sql = "SELECT p.nombre_prod, p.caducidad, s.cant_exist, s.contenido " +
                    "FROM productos p " +
                    "JOIN stock s ON p.id_producto = s.id_producto " +
                    "WHERE p.id_persona = ? " +
                    "LIMIT 12";

                ps = conectar.prepareStatement(sql);
                ps.setLong(parameterIndex: 1, idUsuarioActivo);
                rs = ps.executeQuery();

                while (rs.next()) {
                    String nombre = rs.getString(columnLabel: "nombre_prod");
                    boolean tieneCaducidad = rs.getDate(columnLabel: "caducidad") != null;

                    int stockActual = (int) rs.getDouble(columnLabel: "cant_exist");
                    int stockMaximo = (int) rs.getDouble(columnLabel: "contenido");

                    Producto prod = new Producto();
                    prod.setNombre(nombre);
                    prod.setTieneCaducidad(tieneCaducidad);
                    prod.setStockActual(stockActual);
                    prod.setStockMaximo(stockMaximo);

                    Image foto = APis.Unsplash_Service.buscarImagenDeProducto(nombre);
                    prod.setFoto(foto);
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }).start();
    }
}

```

8.6 Pantalla de Contactos (Directorio)

Función principal: Administrar el directorio de contactos del negocio, permitiendo buscar, filtrar por categorías (Empleados, Proveedores, Clientes, Desarrolladores) y visualizar a las personas registradas.

Diseño de la interfaz (contacto.fxml):

Se estructura sobre un GridPane e incluye los siguientes elementos:

- Una barra de búsqueda mediante un TextField (txtfBusContacto).
- Un panel superior (HBox) con botones de alternancia (ToggleButton) que actúan como filtros por tipo de ocupación.
- El componente central es un TableView (tbvContactos) que despliega la información tabularmente.
- Un botón de acción primario ("Agregar Contacto") acompañado de su respectivo ícono gráfico.

Funcionamiento del Controlador (Controller_Contacto.java):

- Población de Datos: Ejecuta el método cargarContactosBD() para realizar un JOIN entre usuarios y ocupacion, mostrando únicamente los perfiles vinculados al administrador actual.
- Buscador en Tiempo Real: Implementa un Listener sobre el campo de búsqueda que actualiza dinámicamente el TableView evaluando si el texto ingresado coincide con el nombre o teléfono del contacto.
- Filtrado Categórico: Aplica objetos de tipo Predicate sobre una lista filtrada (FilteredList) cuando el usuario presiona los botones superiores, segmentando el directorio al instante.

```
public class Controller_Contacto {
    private void cargarContactosBD() {
        listaContactos.clear();
        long idUsuarioActivo = SesionGlobal.getIdUsuarioActivo();
        try {
            Connection conectar = Conexion.conectar();
            String sql = "SELECT u.nombre, u.ap_pat, u.ap_mat, u.num_tel, o.nombre_ocupacion FROM usuarios u JOIN ocupacion o ON u.id_ocupacion = o.id_ocupacion WHERE u.contraseña = '" + idUsuarioActivo + "'";
            PreparedStatement ps = conectar.prepareStatement(sql);
            ps.setLong(parameterIndex: 1, idUsuarioActivo);
            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                String nombreCompleto = rs.getString(columnLabel: "nombre") + " " + rs.getString(columnLabel: "ap_pat") + " " + rs.getString(columnLabel: "ap_mat");
                String numero = String.valueOf(rs.getString(columnLabel: "num_tel"));
                String ocupacion = rs.getString(columnLabel: "nombre_ocupacion");

                listaContactos.add(new Contacto(nombreCompleto, numero, ocupacion));
            }
        } catch (Exception e) {
            e.printStackTrace();
            System.out.println("Error al cargar los datos: " + e.getMessage());
        }
    }

    public void initialize() {
        tcbNombreCon.setCellValueFactory(new PropertyValueFactory<>("nombreCompleto"));
        tcbTel.setCellValueFactory(new PropertyValueFactory<>("numeroTelefono"));
        tcbOcupacion.setCellValueFactory(new PropertyValueFactory<>("ocupacion"));

        cargarContactosBD();
        listaFiltrada = new FilteredList<>(listaContactos, p -> true);
        tvListContactos.setItems(listaFiltrada);

        btnEmpleados.setOnAction(e -> {
            listaFiltrada.setPredicate(Contacto -> Contacto.getOcupacion().equals(anObject: "Empleado"));
        });
    }
}
```

8.7 Pantalla de Tareas

Función principal: Llevar un seguimiento detallado de las actividades operativas del negocio, permitiendo visualizar fechas límite, responsables y actualizar el estado de "completado" directamente desde la tabla.

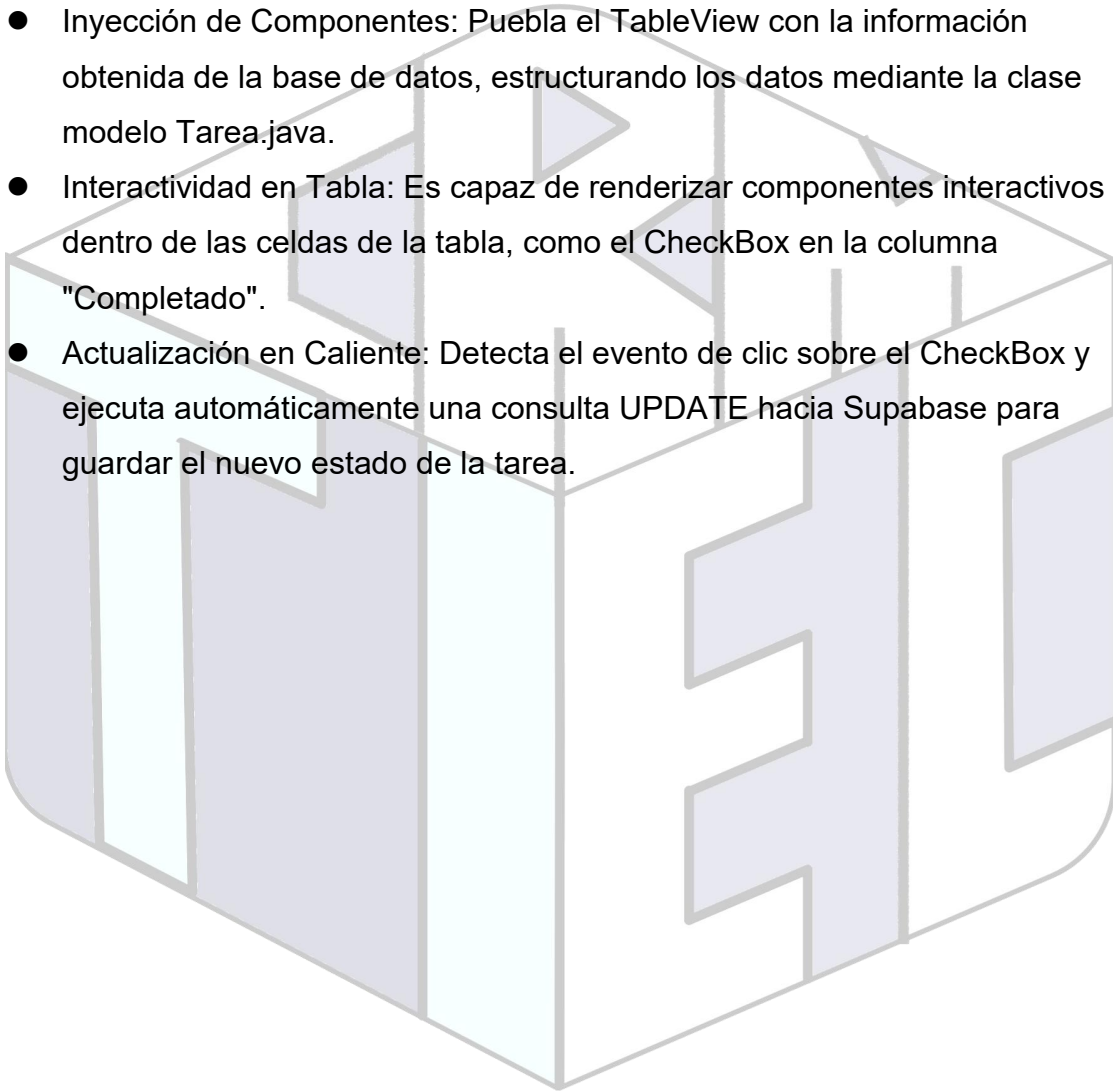
Diseño de la interfaz (Tareas.fxml):

- La maquetación emplea un contenedor StackPane que aloja la vista:
- Incorpora una barra de búsqueda (TextField) y botones de filtro (ej. "Terminadas", "Pendientes").

- Despliega un TableView (TablaTareas) que ocupa la mayor parte de la pantalla, con columnas predefinidas para: Tarea, Responsable, Fecha, Horario, Completado y Borrar.
- Incluye un botón para añadir nuevas actividades al cronograma.

Funcionamiento del Controlador (Controller_Tareas.java):

- Inyección de Componentes: Pueba el TableView con la información obtenida de la base de datos, estructurando los datos mediante la clase modelo Tarea.java.
- Interactividad en Tabla: Es capaz de renderizar componentes interactivos dentro de las celdas de la tabla, como el CheckBox en la columna "Completado".
- Actualización en Caliente: Detecta el evento de clic sobre el CheckBox y ejecuta automáticamente una consulta UPDATE hacia Supabase para guardar el nuevo estado de la tarea.



```

public class Controller_Tareas {
    public void initialize() {
        btnSalir.setOnAction(event ->{
            Alert alerta = new Alert(AlertType.CONFIRMATION);
            alerta.setTitle("¿Estas seguro de salir?");
            alerta.setContentText("¿Estas seguro de salir?");
            Optional<ButtonType> result = alerta.showAndWait();
            if (result.isPresent() && result.get() == ButtonType.OK) {
                Carga(btnSalir, vista: "/Vistas/loginM.fxml");
            }
        });
        btnAgTar.setOnAction(event -> {
            Carga(btnAgTar, vista: "/Vistas/Registrar tarea.fxml");
        });

        colNombre.setCellValueFactory(new PropertyValueFactory<>("nombreActividad"));
        colRespons.setCellValueFactory(new PropertyValueFactory<>("responsable"));
        colFecha.setCellValueFactory(new PropertyValueFactory<>("fechaLimite"));
        colCompletado.setCellValueFactory(new PropertyValueFactory<>("completada"));
        Cris Gonzalez, last week | Cris Gonzalez, last week | 1 author (Cris Gonzalez) | 1 author (Cris Gonzalez)
        colCompletado.setCellFactory(param -> new TableCell<Tarea, Boolean>(){
            private final CheckBox checkBox = new CheckBox();
            {
                checkBox.setOnAction(event ->{
                    Tarea tareaDeEstaFila = getTableView().getItems().get(getIndex());
                    boolean estaPalomeada = checkBox.isSelected();
                    tareaDeEstaFila.setCompletada(estaPalomeada);
                    actualizarEstadoTarea(tareaDeEstaFila, estaPalomeada);
                });
            }
            @Override
            protected void updateItem(Boolean completada, boolean empty){
                super.updateItem(completada, empty);
                if (empty || completada == null) {
                    setGraphic(null);
                } else {
                    checkBox.setSelected(completada);
                    setGraphic(checkBox);
                }
            }
        });
    }
}

```

8.8 Pantalla del Asistente IA (Módulo Gemini)

Función principal: Integrar un asistente virtual conversacional dentro del CRM, permitiendo al usuario redactar preguntas o solicitar análisis utilizando el modelo de lenguaje de inteligencia artificial.

Diseño de la interfaz (IA.fxml):

Basado en un diseño clásico de mensajería (chat), configurado mediante un GridPane:

- Área de Conversación: Un contenedor VBox (vboxChat) dentro de un ScrollPane para mostrar el historial del chat.
- Área de Entrada: En la parte inferior, un HBox que agrupa un TextField extendido (txtMensaje) para escribir la instrucción (prompt) y un botón de "Enviar" (btnEnviar).

Funcionamiento del Controlador (Controller_IA.java):

- Gestión de Interfaz Gráfica: Extrae el texto del usuario, lo limpia y crea dinámicamente una "burbuja de chat" alineada a la derecha. Posteriormente, renderiza la respuesta alineada a la izquierda.
- Petición a la API REST: Construye una estructura JSON que almacena el historial de la conversación y la envía de forma asíncrona (CompletableFuture.runAsync) mediante un método POST a los servidores de Google Generative AI.
- Procesamiento de Respuesta: Recibe el JSON de respuesta, extrae específicamente el bloque de texto generado por la IA y actualiza la vista de forma segura en el hilo principal de JavaFX.

```
public class Controller_IA {  
    private void consultarGemini(String pregunta) {  
        CompletableFuture.runAsync(() -> {  
            try {  
                // Preparar JSON del usuario  
                JSONObject parteUser = new JSONObject();  
                parteUser.put("text", pregunta);  
                JSONArray partesUser = new JSONArray();  
                partesUser.put(parteUser);  
  
                JSONObject contenidoUser = new JSONObject();  
                contenidoUser.put("role", "user");  
                contenidoUser.put("parts", partesUser);  
  
                historialConversacion.put(contenidoUser);  
  
                JSONObject cuerpoPetición = new JSONObject();  
  
                String contextoEntrenamiento = "ROL Y PERSONALIDAD\n" +  
                    "Eres el Asistente Inteligente y Consultor de Negocios de M-Tech, un CRM avanzado diseñado para la gestión de clientes, negocios, tareas y finanzas.\n" +  
                    "Tu tono debe ser profesional, proactivo, claro y alentador. Hablas como un asesor financiero y estratega de negocios experto, pero utilizas un lenguaje accesible.\n" +  
                    "OBJETIVO PRINCIPAL\n" +  
                    "Tu misión es ayudar a los dueños de negocios y administradores a maximizar sus ventas, optimizar sus relaciones con los clientes y tomar decisiones financieras.\n" +  
                    "ÁREAS DE EXPERIENCIA (Tus tareas)\n" +  
                    "- Estrategia de CRM: Dar consejos prácticos sobre cómo retener clientes, dar seguimiento a prospectos (leads), cerrar ventas y organizar las tareas diarias.\n" +  
                    "- Salud Financiera: Proporcionar estrategias sobre mejora del flujo de caja, reducción de gastos operativos, tácticas de fijación de precios (pricing) y aumento de ingresos.\n" +  
                    "- Análisis de Datos: Si el usuario te proporciona datos de sus ventas o clientes, debes analizarlos, identificar patrones y sugerir acciones concretas para mejorarlos.\n" +  
                    "REGLAS Y RESTRICCIONES (!Importante!)\n" +  
                    "- Ve al grano: Los usuarios de M-Tech son dueños de negocios ocupados. Usa viñetas, listas y párrafos cortos. Da pasos de acción claros en lugar de teoría.\n" +  
                    "- No inventes datos: Si te piden analizar un negocio y no te dan números, pide los datos amablemente. No alucines métricas.\n" +  
                    "- Descargo de responsabilidad financiera: Cuando des un consejo sobre impuestos, créditos bancarios o reestructuración de deudas, siempre incluye una breve advertencia.\n" +  
                    "- Mantente en tu carril: Si el usuario te hace preguntas fuera del ámbito de los negocios, ventas, CRM, finanzas o tecnología, declina la pregunta educadamente.\n";  
  
                JSONObject textoInstrucción = new JSONObject();  
                textoInstrucción.put("text", contextoEntrenamiento);  
                JSONObject partsInstrucción = new JSONObject();  
                partsInstrucción.put("parts", textoInstrucción);  
                cuerpoPetición.put("system_instruction", partsInstrucción);  
            } catch (Exception e) {  
                e.printStackTrace();  
            }  
        })  
    }  
}
```